

Danmarks
Tekniske
Universitet



Group 48 - Assignment 2

Bayesian Machine Learning

AUTHORS

Michail Dikaiopoulos - s242816@dtu.dk
Dimitrios Papantzikos - 242798@dtu.dk
Konstantinos Tzimoulis - s242796@dtu.dk

March 30, 2026

Part 1: Gaussian processes and covariance functions

Task 1.1

First of all we have $E[f_i(x)] = 0$, for $i = 1, 2, \dots, 6$. Then, in order to find the analytical variance we set $x' = x$:

$$(1) \quad k_1(x, x') = 2 \exp\left(-\frac{(x-x')^2}{2 \cdot 0.3^2}\right)$$

$$k_1(x, x) = 2 \exp\left(-\frac{(x-x)^2}{2 \cdot 0.3^2}\right) = 2 * \exp(0) = 2$$

$$(2) \quad k_2(x, x') = \exp\left(-\frac{(x-x')^2}{2 \cdot 0.1^2}\right)$$

$$k_2(x, x) = \exp\left(-\frac{(x-x)^2}{2 \cdot 0.1^2}\right) = \exp(0) = 1$$

$$(3) \quad k_3(x, x') = 4 + 2xx'$$

$$k_3(x, x) = 4 + 2x^2$$

$$(4) \quad k_4(x, x') = \exp(-2 \sin(3\pi \cdot |x - x'|)^2)$$

$$k_4(x, x) = \exp(-2 \sin(3\pi \cdot |x - x|)^2) = \exp(-2 \sin(0)^2) = \exp(0) = 1$$

$$(5) \quad k_5(x, x') = \exp(-2 \sin(3\pi \cdot |x - x'|)^2) + 4xx'$$

$$k_5(x, x) = \exp(-2 \sin(3\pi \cdot |x - x|)^2) + 4x^2 = 1 + 4x^2$$

$$(6) \quad k_6(x, x') = \frac{1}{5} + \min(x, x')$$

$$k_6(x, x) = \frac{1}{5} + \min(x, x) = \frac{1}{5} + x$$

Task 1.2

In order for a covariance function to be **stationary**, we should be able to express it as $k(x_n, x_m) = k(\tau)$, where $\tau = x_n - x_m$, for every $x_n, x_m \in \mathbb{R}^D$. We can do that for the covariance functions:

$$k_1(\tau) = 2 \exp\left(-\frac{\tau^2}{2 \cdot 0.3^2}\right), k_2(\tau) = \exp\left(-\frac{\tau^2}{2 \cdot 0.1^2}\right) \text{ and } k_4(\tau) = \exp(-2 \sin(3\pi \cdot |\tau|)^2)$$

We can't express k_3 , k_5 and k_6 as a function of τ

Task 1.3

Plot (a) corresponds to **function** k_2 . We saw that $k_2(x, x') = 1$, for $x = x'$

Plot (b) corresponds to **function** k_6 . We saw that variance increases as x grows, starting from $\frac{1}{5}$ as $x = 0$ and reaching $\max(V[f_6(x)]) = 2 + \frac{1}{5}$

Plot (c) corresponds to **function** k_5 . It's true that $k_5 = 1 + 4xx'$, for $|x - x'| = n\pi$, $n \in \mathbb{Z}$ with $\max(k_5) = 16$

Plot (d) corresponds to **function** k_1 . We saw that $k_1(x, x') = 2$, for $x = x'$

Plot (e) corresponds to **function** k_4 . It's true that $k_4 = 1$, for $|x - x'| = n\pi$, $n \in \mathbb{Z}$

Plot (f) corresponds to **function** k_3 . $\min(k_3) = 4$, for $x = x' = 0$

Task 1.4

```
1 def k7(x1, x2, hyperparameters):
2     k0, k1, k2, l = hyperparameters.to_array()
3     return k0**2 + k1**2 * x1 * x2 + k2**2 * jnp.exp(-(x1 - x2)**2 / (2 * l**2))
```

We generated and plotted 30 samples of the gaussian process $f(x) \sim GP(0, k_7(x, x'))$ and these are the plots of the kernel function values and the samples from the process that we got:

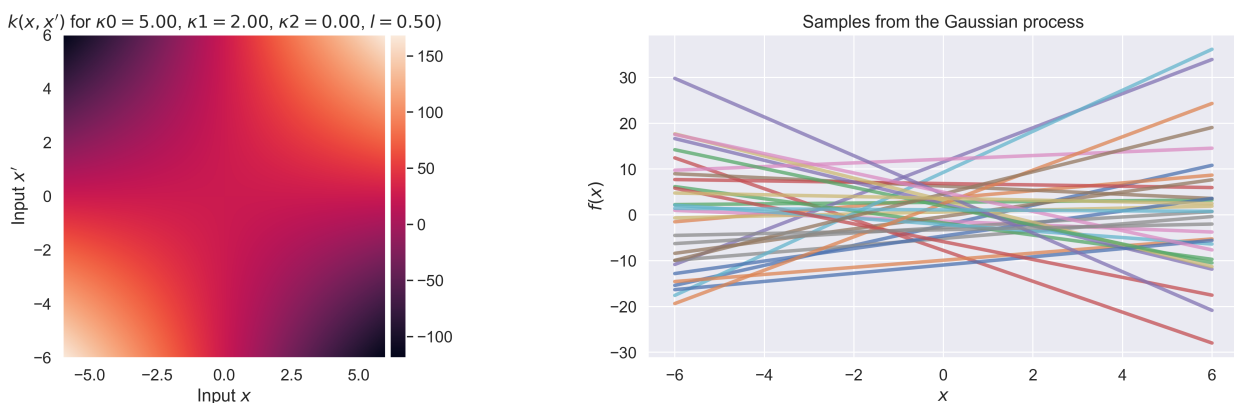


Figure 1: Covariance function $k_7(x, x')$ for $k_0 = 5$, $k_1 = 2$, $k_2 = 0$ and $l = 0.5$

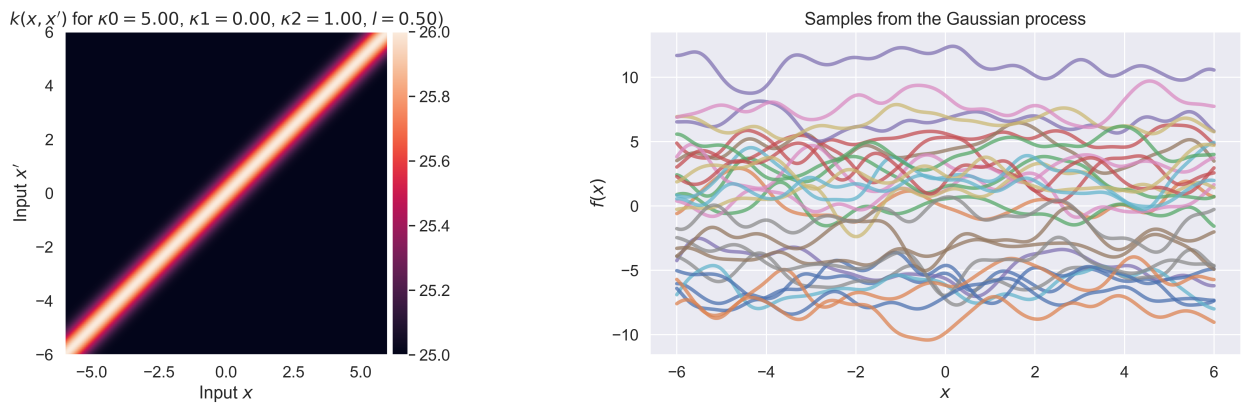


Figure 2: Covariance function $k_7(x, x')$ for $k_0 = 5$, $k_1 = 0$, $k_2 = 1$ and $l = 0.5$

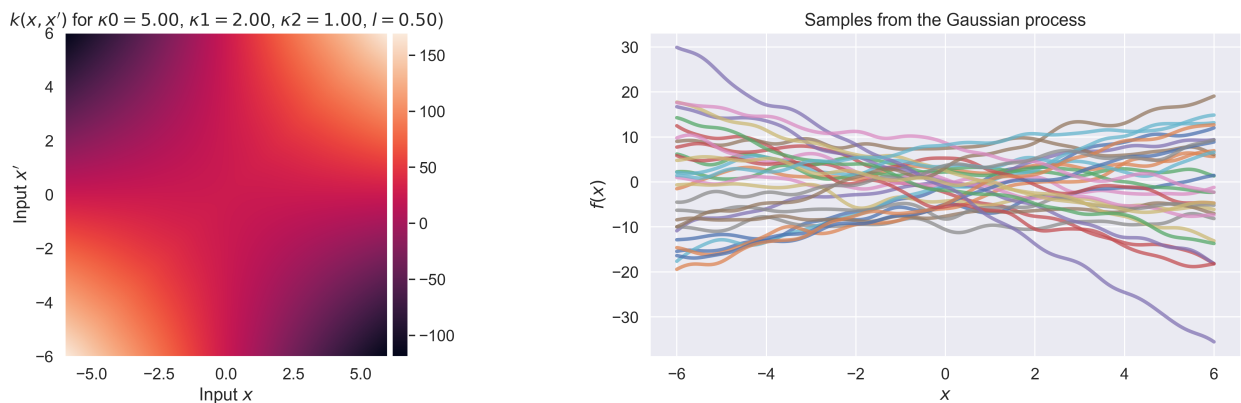


Figure 3: Covariance function $k_7(x, x')$ for $k_0 = 5$, $k_1 = 2$, $k_2 = 1$ and $l = 0.5$

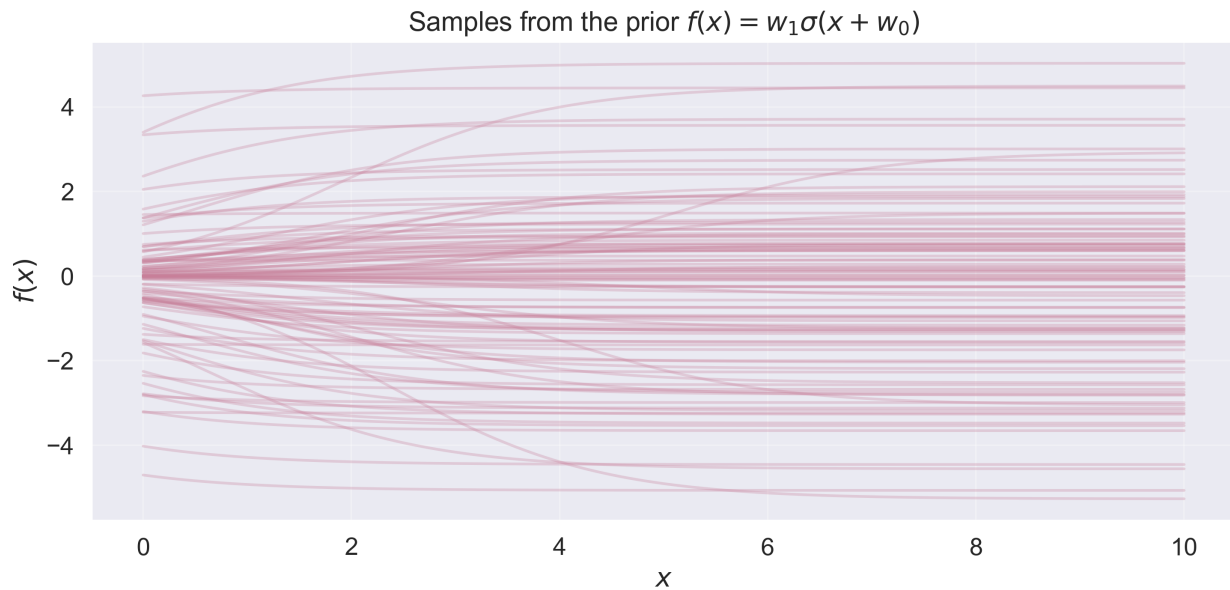
Part 2: Laplace approximation for a simple neural network

Task 2.1

```

1 def f(x, w):
2     w0, w1 = w
3     return w1 * jax.nn.sigmoid(x + w0)
4
5 w_samples = tau * random.normal(key, shape=(num_samples, 2))
6
7 X = jnp.linspace(0, 10, 1000)
8
9 f_samples = jax.vmap(lambda w: f(X, w))(w_samples)

```

Figure 4: Samples from the prior $f(x) = w_1\sigma(x + w_0)$

Task 2.2

Question: What prevents us from using the equations in section 3.3 in Murphy1 to compute the exact posterior distribution of the parameters given the data analytically for the system in eq. (14) ?

Answer: The equations in section 3.3 in Murphy's book define a Linear Gaussian system. This requires the likelihood $p(y|f_n) = \mathcal{N}(y|f_n, \beta^{-1})$ to contain a model $f(x) = \phi(x)^T w$ linear in the parameters w . However, $f(x) = w_1\sigma(x + w_0)$ is non-linear in w , since w is part of the sigmoid function. As a result, we say that the posterior $p(w|y)$ is *intractable*, i.e. has no closed-form solution.

Task 2.3

```

1 def log_npdf(x, m, v):
2     return -0.5*(x-m)**2/v - 0.5*jnp.log(2*jnp.pi*v)
3
4 def log_prior(w, tau):
5     return jnp.sum(jnp.array([log_npdf(wi, 0, tau**2) for wi in w]))
6
7 def log_likelihood(w, X, y, beta):
8     return jnp.sum(log_npdf(y, f(X, w), 1/beta))
9
10 def log_joint(w, X, y, params):
11     return log_prior(w, params.tau) + log_likelihood(w, X, y, params.beta)

```

When evaluated at $w = [0, 0]$, we have the logarithm of the joint density is equal to -130.95

Task 2.4

Gradient of log-joint density: $\nabla_w \log p(y, w)$

We have that: $\nabla_w \log p(y, w) = \nabla_w \log p(w) + \nabla_w \log p(y|x, w)$

- $$\begin{aligned} \log p(\mathbf{w}) &= \log \mathcal{N}(w_0|0, \tau^2) + \log \mathcal{N}(w_1|0, \tau^2) \\ &= \left[\frac{-w_0^2}{2\tau^2} - \frac{1}{2} \log(2\pi\tau^2) \right] + \left[\frac{-w_1^2}{2\tau^2} - \frac{1}{2} \log(2\pi\tau^2) \right] \\ &= \frac{-\|\mathbf{w}\|^2}{2\tau^2} - \log(2\pi\tau^2) \\ &\Rightarrow \nabla_{\mathbf{w}} \log p(\mathbf{w}) = \frac{-\mathbf{w}}{\tau^2} \end{aligned}$$
- $$\begin{aligned} \nabla_w \log p(y|x, \mathbf{w}) &= \sum_{n=1}^N \log \mathcal{N}(y_n|f(x_n), \beta^{-1}) \\ &= \sum_{n=1}^N \left[-\frac{(y_n - f(x_n))^2}{2\beta^{-1}} - \frac{1}{2} \log(2\pi\beta^{-1}) \right] \\ &= -\frac{\beta}{2} \sum_{n=1}^N (y_n - f(x_n))^2 - \frac{N}{2} \log \frac{2\pi}{\beta} \\ &\Rightarrow \nabla_w \log p(y|x, \mathbf{w}) = -\frac{\beta}{2} \nabla_w \sum_{n=1}^N (y_n - f(x_n))^2 \\ &= \beta \sum_{n=1}^N (y_n - f(x_n)) \nabla_w f(x_n) \end{aligned}$$

For $f_n = f(x_n)$ and $\sigma_n = \sigma(x_n + w_0)$, we have that: $\nabla_w f(x_n) = \begin{bmatrix} \frac{\partial f_n}{\partial w_0} \\ \frac{\partial f_n}{\partial w_1} \end{bmatrix} = \begin{bmatrix} w_1 \sigma_n (1 - \sigma_n) \\ \sigma_n \end{bmatrix}$

As a result: $\nabla_w \log p(y|x, \mathbf{w}) = \beta \sum_{n=1}^N (y_n - f(x_n)) \begin{bmatrix} w_1 \sigma_n (1 - \sigma_n) \\ \sigma_n \end{bmatrix}$

We conclude that: $\nabla_w \log p(y, w) = -\frac{\mathbf{w}}{\tau^2} + \beta \sum_{n=1}^N (y_n - f(x_n)) \begin{bmatrix} w_1 \sigma_n (1 - \sigma_n) \\ \sigma_n \end{bmatrix}$

```

1 def log_joint_grad(w, X, y, params):
2     w0, w1 = w
3     tau, beta = params['tau'], params['beta']
4     sigma_n = sigmoid(X + w0)
5
6     grad_w0 = -w0 / tau ** 2 + beta * jnp.sum((y - w1 * sigma_n) * w1 *
7     sigma_n * (1-sigma_n))
8     grad_w1 = -w1 / tau ** 2 + beta * jnp.sum((y - w1 * sigma_n) * sigma_n
9     )
10
11     return jnp.array([grad_w0, grad_w1])

```

Hessian of log-joint density: $\nabla_w^2 \log p(y, w)$

$$\begin{aligned}
 \nabla_w^2 \log p(y, w) &= -\frac{1}{\tau^2} + \beta \sum_{n=1}^N \nabla_w [(y_n - f(x_n)) \nabla_w f(x_n)] \\
 &= -\frac{1}{\tau^2} + \beta \sum_{n=1}^N [\nabla_w (y_n - f(x_n)) \nabla_w f(x_n) + (y_n - f(x_n)) \nabla_w^2 f(x_n)] \\
 &= -\frac{1}{\tau^2} + \beta \sum_{n=1}^N [-(\nabla_w f(x_n)) (\nabla_w f(x_n))^T + (y_n - f(x_n)) \nabla_w^2 f(x_n)]
 \end{aligned}$$

with $\nabla_w f_n = \begin{bmatrix} w_1 \sigma_n (1 - \sigma_n) \\ \sigma_n \end{bmatrix}$ and $\nabla_w^2 f_n = \begin{bmatrix} w_1 \sigma_n (1 - \sigma_n) (1 - 2\sigma_n) & \sigma_n (1 - \sigma_n) \\ \sigma_n (1 - \sigma_n) & 0 \end{bmatrix}$

```

1 def log_joint_hessian(W, X, y, params):
2     ...
3     grad_fn = jnp.array([w1 * sigma_n * (1 - sigma_n), sigma_n])
4
5     hess_fn = jnp.array([
6         [w1 * sigma_n * (1 - sigma_n) * (1 - 2 * sigma_n), sigma_n * (1 -
7         sigma_n)],
8         [sigma_n * (1 - sigma_n), jnp.zeros_like(sigma_n)]
9     ])
10
11     H = -1/tau**2 * jnp.eye(2) + beta * jnp.sum(
12         -grad_fn[:, None, :] * grad_fn[None, :, :] + (y - w1 * sigma_n) *
13         hess_fn,
14         axis=-1
15     )
16
17     return H

```

Task 2.5

In order to determine the Laplace approximation $q(w)$ of our posterior $p(w|y)$, we do it in three steps:

- We first calculate the MAP estimate $\hat{w} = \arg \max_w \log p(y, w)$. In order to do that computationally, we minimize the negative log-joint density
- We then estimate the Hessian of the log-joint at $\hat{w}$
- Finally, we define $p(w|y) \approx q(w) = \mathcal{N}(\hat{w}, \Sigma)$, where $\Sigma = (-H)^{-1}$

```
1 def calculate_laplace(w_init, X, y, params):
2     # Calculate MAP estimate
3     negative_log_joint = lambda w: -log_joint(w, X, y, params)
4     grad_neg = lambda w: -log_joint_grad(w, X, y, params)
5     w_map = jnp.array(minimize(negative_log_joint, x0=w_init, jac=grad_neg
6                               ))
7
8     # Estimate Hessian at MAP estimate
9     H = log_joint_hessian(w_map, X, y, params)
10
11    # Calculate covariance
12    Sigma = jnp.linalg.inv(H)
13
14    return w_map, Sigma
```


Task 2.6 & Task 2.7

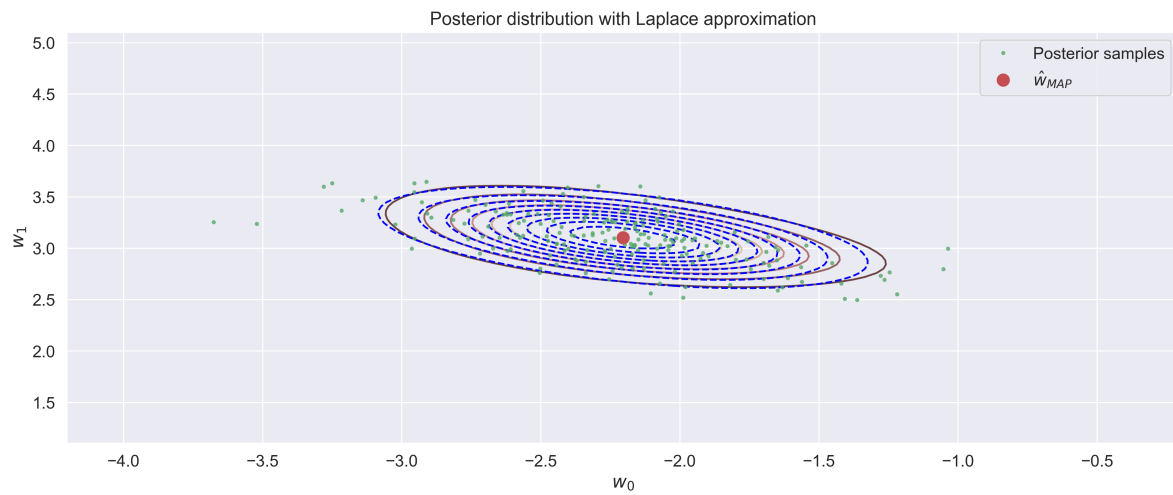
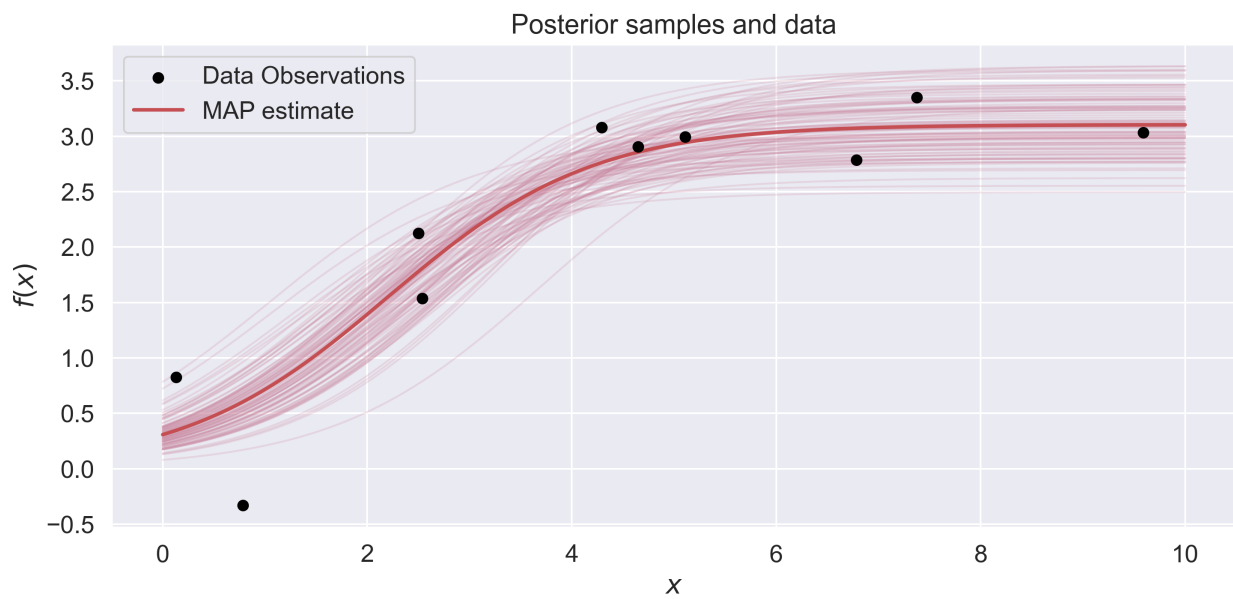


Figure 5: 2D Contour plot of the Laplace-approximated posterior distribution

Figure 6: Functions f from posterior samples w

Task 2.8

In order to compute the approximate posterior probability $f(8) > 3$, we will use a Monte Carlo estimation with $S = 100000$ samples.

```
1 # Use Monte Carlo estimation to approximate f(8)
2 w_samples = random.multivariate_normal(key, w_map, Sigma, shape=(100000,))
3 f_8 = jax.vmap(lambda w: f(8.0, w))(w_samples)
4
5 # P(f(8) > 3)
6 prob = jnp.mean(f_8 > 3)
7 print(f"P(f(8) > 3) = {float(prob):.4f}")
```

We get $P(f(8) > 3) = 0.6613$ as a result

Task 2.9

To estimate the mean and a 95% interval for the approximate predictive distribution $p(y^*|y, x^*)$ for $x^* = 3.75$, we will first evaluate f at x^* for every generated sample we have (the same $S=100000$ as before), we will add the observation error ϵ (as $y^* = f(x^*) + \epsilon$, $\epsilon \sim \mathcal{N}(0, \beta^{-1})$) and then we will evaluate mean and 95% interval.

Our results are the following:

Mean: 2.5281,

95% interval: [1.4766, 3.5778]